ĐẠI HỌC BÁCH KHOA – ĐHQG TP.HCM
Khoa Khoa học và Kỹ thuật Máy tính

Bài tập lớn A1

CNN vs Transformer trên CIFAR-100: from-scratch, pretrained và các kiểu đóng băng

Học sâu và ứng dụng trong thị giác máy tính – CO5085

Học kỳ: HK261 (2026–2027)
Giảng viên: Lê Thành Sách, Nguyễn Quốc Minh
Học viên: Trần Hoài Tâm – MSHV 2470743 (làm cá nhân)
Mã nguồn: github.com/TamTran9x/2470743-CO5085-Assignment/A1
Trang A1: tamtran9x.github.io/2470743-CO5085-Assignment/A1

TP. Hồ Chí Minh, tháng 11/2026

Mục lục

1 Giới thiệu . . . 3
2 Tập dữ liệu (A1.1) . . . 3
2.1 Lựa chọn và giấy phép . . . 3
2.2 Chia tập (split) . . . 3
2.3 Phân tích khám phá (EDA) . . . 4
3 DataLoader và augmentation . . . 5
4 Mô hình và chế độ huấn luyện . . . 6
4.1 Hai kiến trúc . . . 6
4.2 Bốn chế độ huấn luyện . . . 7
5 Protocol thí nghiệm . . . 7
6 Kết quả . . . 8
6.1 Nhận xét chính . . . 9
6.2 Chênh lệch CNN–ViT có ý nghĩa thống kê không? . . . 9
6.3 Phân tích lỗi . . . 10
7 Thảo luận . . . 13
7.1 Pretrained so với from scratch . . . 13
7.2 Khi nào ViT cần pretraining? . . . 13
7.3 Khi nào đóng băng có lợi? . . . 13
7.4 Chi phí . . . 14
8 Hạn chế và hướng phát triển . . . 14
9 Tái lập kết quả . . . 15
10 Tài liệu tham khảo . . . 15

1 Giới thiệu

Bài tập so sánh hai họ kiến trúc phân loại ảnh, mạng tích chập (CNN) và Vision Transformer (ViT), trên một tập ảnh đủ lớn và khó là **CIFAR-100**. Mỗi kiến trúc được chạy ở bốn chế độ huấn luyện: (i) huấn luyện từ đầu (from scratch); (ii) dùng trọng số pretrained trên ImageNet-1k và đóng băng toàn bộ backbone, chỉ học lớp phân loại (linear probe); (iii) pretrained, đóng băng nửa đầu mạng; (iv) pretrained, fine-tune toàn bộ. Tổng cộng 8 cấu hình chạy trên cùng một split, cùng pipeline dữ liệu và cùng cách đo. Báo cáo trình bày lựa chọn dataset và protocol (mốc A1.1), sau đó là kết quả, chi phí, phân tích lỗi và thảo luận (mốc A1.2).

Mọi con số trong báo cáo được sinh tự động từ `results/` bởi mã nguồn trong thư mục A1. Bảng biểu được đọc trực tiếp từ `results/summary.csv` khi biên dịch, nên báo cáo luôn khớp với lần chạy thật.

2 Tập dữ liệu (A1.1)

2.1 Lựa chọn và giấy phép

CIFAR-100 (Krizhevsky, 2009) gồm 50000 + 10000 = 60.000 ảnh màu 32×32, thuộc 100 lớp (fine label), được gom thành 20 superclass (coarse label), mỗi superclass có đúng 5 lớp. Mỗi lớp có 500 ảnh train và 100 ảnh test, tức dữ liệu cân bằng hoàn toàn.

- **Nguồn:** bản gốc tại `cs.toronto.edu/~kriz/cifar.html`. Mã nguồn tải bản mirror chính thức `uoft-cs/cifar100` trên Hugging Face Hub (định dạng parquet, cùng ảnh và cùng split train/test), vì máy chủ gốc tải rất chậm (~10 KB/s khi thực hiện).
- **Giấy phép:** tác giả không công bố license chính thức; dataset được cung cấp công khai cho nghiên cứu và giảng dạy, là một trong các benchmark phổ biến nhất (dataset card trên Hugging Face ghi `license: unknown`). Ảnh được trích từ tập 80 Million Tiny Images; tập mẹ này đã bị rút lại năm 2020, nhưng CIFAR-10/100 vẫn được duy trì và dùng rộng rãi. Bài tập chỉ dùng dữ liệu cho mục đích học tập.
- **Lý do chọn:** (1) có trong danh sách gợi ý của đề và không thuộc nhóm bị cấm (MNIST, Fashion-MNIST, CIFAR-10); (2) 100 lớp, nhiều lớp rất gần nhau trong cùng superclass (maple/oak/willow tree, boy/girl/man/woman…), khó hơn nhiều so với CIFAR-10; (3) ảnh 32×32 khác xa phân phối ImageNet 224×224, tạo ra **domain gap** để thấy rõ khác biệt giữa đóng băng và fine-tune; (4) toàn bộ tập nằm gọn trong bộ nhớ GPU nên chạy đủ 8 cấu hình ở độ phân giải 224 trên GPU laptop 6 GB.

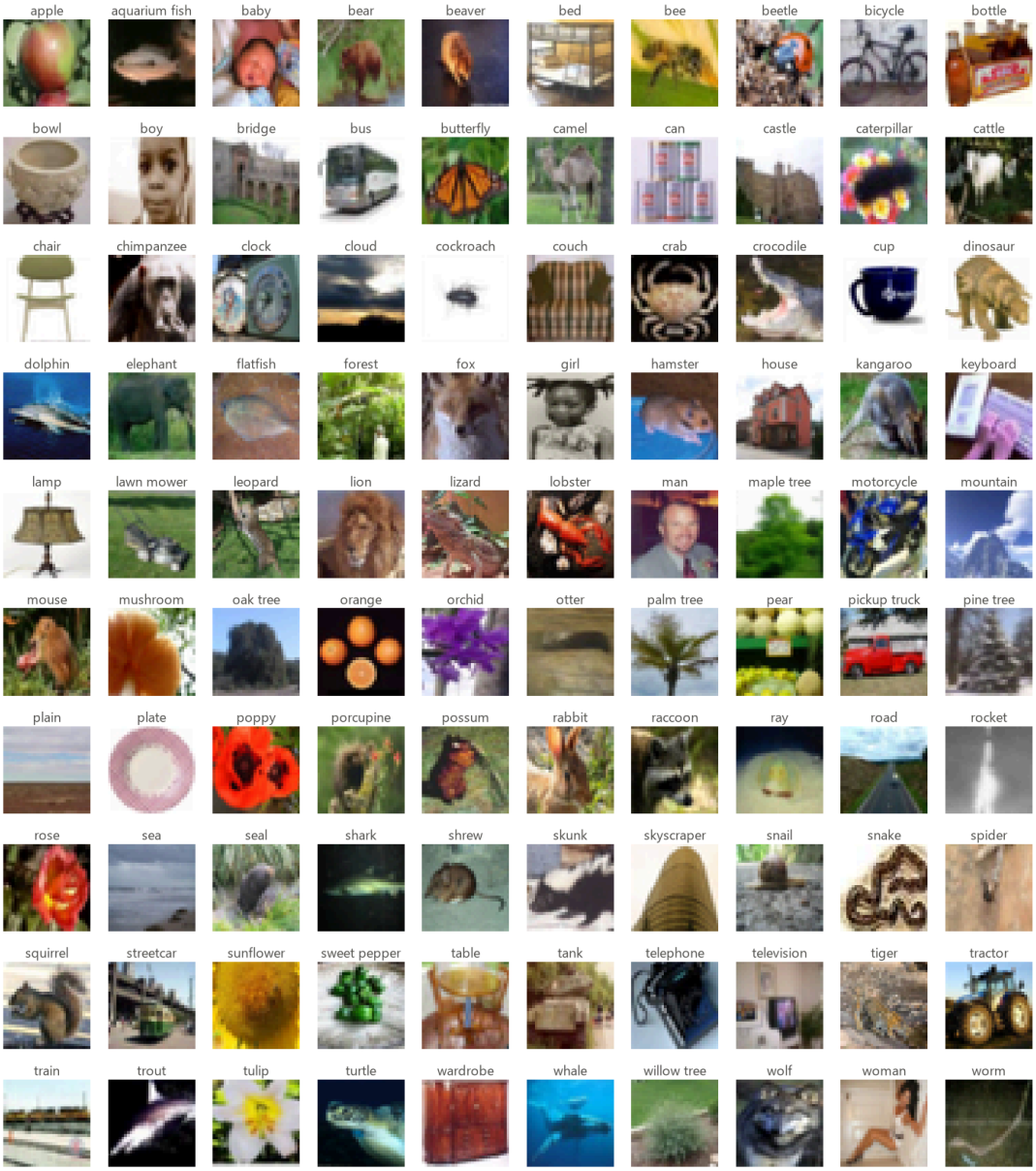
2.2 Chia tập (split)

Tập	Số ảnh	Ảnh/lớp	Vai trò
Train	45000	450	Huấn luyện
Validation	5000	50	Chọn epoch tốt nhất, so sánh khi huấn luyện
Test (chính thức)	10000	100	Chỉ đánh giá một lần cho checkpoint đã chọn

Bảng 1: Chia tập. Validation được tách **phân tầng** (50 ảnh/lớp, seed 0) từ 50.000 ảnh train chính thức.

Tập test chính thức được giữ riêng: mọi lựa chọn (epoch tốt nhất, chế độ tốt nhất của từng họ) đều dựa trên validation. Split cố định, giống nhau cho cả 8 cấu hình, và tái tạo được bằng `stratified_split(seed=0)` trong `src/data.py`.

2.3 Phân tích khám phá (EDA)



Hình 1: Mỗi lớp một ảnh ngẫu nhiên từ tập train. Ảnh có độ phân giải rất thấp; nhiều lớp chỉ phân biệt được qua chi tiết nhỏ hoặc ngữ cảnh.

- **Cân bằng lớp:** train có đúng 450 ảnh/lớp, validation 50 ảnh/lớp, test 100 ảnh/lớp. Vì vậy accuracy và macro-F1 gần như trùng nhau, và không cần trọng số lớp hay oversampling.
- **Thống kê pixel** (train, thang [0, 1]): mean = (0,507; 0,487; 0,441), std = (0,267; 0,256; 0,276). Giá trị này khác thống kê ImageNet (mean 0,485/0,456/0,406, std 0,229/0,224/0,225), nhưng mọi cấu hình vẫn chuẩn hoá bằng thống kê ImageNet để khớp với các backbone pretrained. Mô hình from-scratch cũng dùng cùng chuẩn hoá cho công bằng; chênh lệch nhỏ này không ảnh hưởng đáng kể vì mạng có BatchNorm/LayerNorm.
- **Cấu trúc phân cấp:** 20 superclass × 5 lớp (Bảng 2). Nhiều lỗi được dự đoán sẽ rơi vào lớp "anh em" trong cùng superclass, điều này được kiểm chứng ở Mục 6.



Hình 2: Phân bố cường độ pixel theo kênh. Đỉnh ở 0 và 255 đến từ nền đen/trắng (ảnh sản phẩm, ảnh cắt nền).

Superclass	5 lớp con
`aquatic_mammals`	beaver, dolphin, otter, seal, whale
`fish`	aquarium fish, flatfish, ray, shark, trout
`flowers`	orchid, poppy, rose, sunflower, tulip
`food_containers`	bottle, bowl, can, cup, plate
`fruit_and_vegetables`	apple, mushroom, orange, pear, sweet pepper
`household_electrical_devices`	clock, keyboard, lamp, telephone, television
`household_furniture`	bed, chair, couch, table, wardrobe
`insects`	bee, beetle, butterfly, caterpillar, cockroach
`large_carnivores`	bear, leopard, lion, tiger, wolf
`large_man-made_outdoor_things`	bridge, castle, house, road, skyscraper
`large_natural_outdoor_scenes`	cloud, forest, mountain, plain, sea
`large_omnivores_and_herbivores`	camel, cattle, chimpanzee, elephant, kangaroo
`medium_mammals`	fox, porcupine, possum, raccoon, skunk
`non-insect_invertebrates`	crab, lobster, snail, spider, worm
`people`	baby, boy, girl, man, woman
`reptiles`	crocodile, dinosaur, lizard, snake, turtle
`small_mammals`	hamster, mouse, rabbit, shrew, squirrel
`trees`	maple tree, oak tree, palm tree, pine tree, willow tree
`vehicles_1`	bicycle, bus, motorcycle, pickup truck, train
`vehicles_2`	lawn mower, rocket, streetcar, tank, tractor

Bảng 2: 20 superclass của CIFAR-100.

3 DataLoader và augmentation

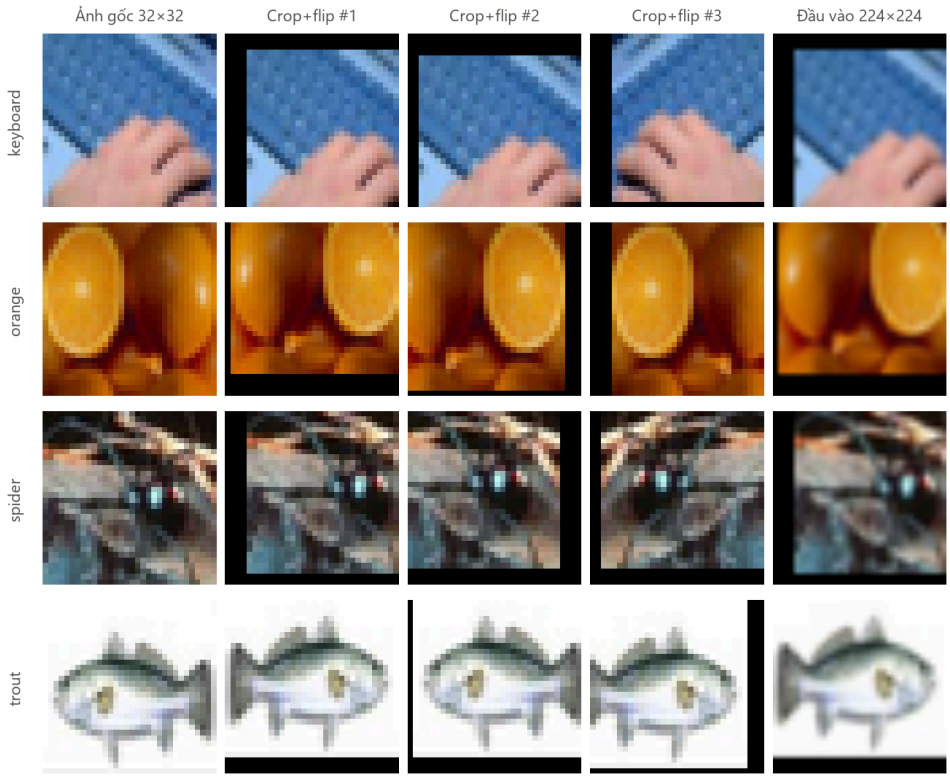
DataLoader đặt trên GPU. Toàn bộ 60.000 ảnh uint8 (~180 MB) được nạp lên GPU một lần (`GPULoader` trong `src/data.py`). Mỗi batch được cắt chỉ số, augment, resize và chuẩn hoá ngay trên GPU. Trên Windows, `DataLoader` đa tiến trình khởi động chậm và giải nén ảnh trên CPU dễ thành nút thắt; cách làm này giữ GPU luôn bận (xem thông lượng ở Bảng 6).

Pipeline (giống hệt cho cả 8 cấu hình):

1. **Train:** random crop 32×32 với zero-padding 4 pixel (độc lập từng ảnh) → lật ngang ngẫu nhiên (p = 0,5).

2. Resize bilinear 32 → **224×224**, đúng độ phân giải mà cả hai backbone pretrained được huấn luyện (DeiT-S patch 16 → 14×14 = 196 token, không cần nội suy position embedding).
3. Chuẩn hoá theo mean/std ImageNet; với CNN, tensor được chuyển sang `channels_last` để tận dụng Tensor Core.
4. **Val/Test:** chỉ resize + chuẩn hoá, không augment.

Augmentation được giữ ở mức cổ điển cho CIFAR (crop + flip) cộng với label smoothing 0,1, và áp dụng **như nhau** cho mọi cấu hình, để khác biệt giữa các cấu hình chỉ đến từ kiến trúc và chế độ huấn luyện. Các kỹ thuật mạnh hơn (RandAugment, Mixup/CutMix) có lợi cho ViT from scratch nhưng sẽ làm lẫn biến số; phần này được bàn ở Mục 8.



Hình 3: Ảnh gốc, ba lần crop+flip ngẫu nhiên, và đầu vào 224×224 thực tế của mô hình (đã khử chuẩn hoá để hiển thị).

4 Mô hình và chế độ huấn luyện

4.1 Hai kiến trúc

Hai mô hình được chọn có kích thước và chi phí tương đương, và đều chỉ được pretrain trên **ImageNet-1k** (không dùng ImageNet-21k), để so sánh công bằng:

Họ	Mô hình (timm)	Tham số	GMACs @224	IN-1k top-1	Đặc điểm
CNN	`resnet50.tv2_in1k`	23,71 M	4,09	80,9%	4 stage bottleneck, BatchNorm
Transformer	`deit_small_patch16_224.fb_in1k`	21,70 M	4,60	79,9%	12 block, dim 384, 6 head, LayerNorm

Bảng 3: Hai kiến trúc (số tham số tính với head 100 lớp; GMACs đo bằng `FlopCounterMode` của PyTorch).

- **ResNet-50** (He và cộng sự, 2016): trọng số torchvision V2, huấn luyện 600 epoch với TrivialAugment, Mixup/CutMix, label smoothing và EMA.

- **DeiT-S** (Touvron và cộng sự, 2021): ViT-Small/16 huấn luyện 300 epoch trên ImageNet-1k với RandAugment, Mixup/CutMix và random erasing (bản không distillation). Hai công thức pretrain đều "hiện đại" và có độ chính xác ImageNet gần nhau.

Head phân loại 1000 lớp được thay bằng một lớp tuyến tính 100 lớp khởi tạo ngẫu nhiên.

4.2 Bốn chế độ huấn luyện

Chế độ	Khởi tạo	ResNet-50: phần đóng băng	DeiT-S: phần đóng băng
From scratch	Ngẫu nhiên	–	–
Freeze backbone	IN-1k	Toàn bộ trừ `fc`	Toàn bộ trừ `head`
Freeze một phần	IN-1k	stem + `layer1`–`layer2` (2/4 stage)	patch-embed, pos-embed, cls, block 0–5 (6/12 block)
Full fine-tune	IN-1k	–	–

Bảng 4: Định nghĩa các chế độ. "Freeze một phần" đóng băng nửa đầu theo độ sâu của mỗi mạng.

Phần bị đóng băng luôn ở chế độ `eval()` khi huấn luyện: BatchNorm của ResNet dùng running statistics của ImageNet và không cập nhật. Autograd không lưu activation và không tính gradient cho phần đóng băng, nên các chế độ freeze vừa nhanh hơn vừa tốn ít bộ nhớ hơn (Bảng 6). Lưu ý: nửa đầu ResNet-50 chỉ chứa khoảng 6% tham số nhưng chiếm khoảng một nửa FLOPs, trong khi nửa đầu DeiT-S chứa khoảng một nửa tham số. Đây là khác biệt cấu trúc giữa hai họ (CNN dồn tham số về các stage sâu, ViT chia đều cho các block).

5 Protocol thí nghiệm

Siêu tham số	From scratch	Freeze backbone	Freeze một phần	Full fine-tune
Epoch	30	10	10	10
LR backbone	5e-4	–	1e-4	1e-4
LR head	5e-4	1e-3	1e-3	1e-3
Warmup (epoch)	3	1	1	1
Chung: AdamW (β = 0,9/0,999), weight decay 0,05 (không áp cho bias, norm, pos-embed, cls token), batch 64, cosine decay theo từng bước sau warmup tuyến tính, label smoothing 0,1, gradient clipping 1,0, mixed precision bfloat16, ảnh 224×224, seed 42.				

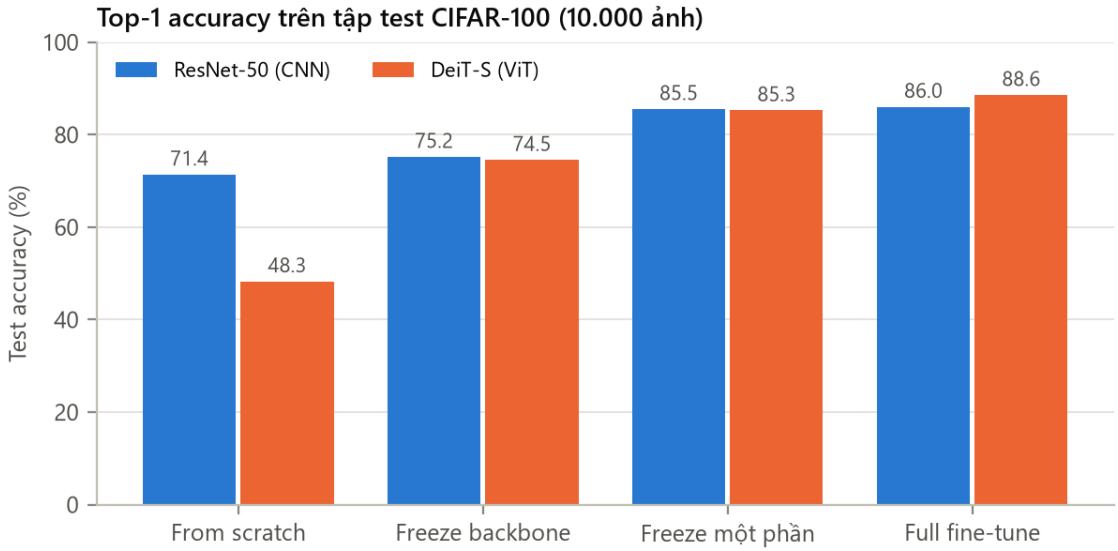
Bảng 5: Siêu tham số. Hai kiến trúc dùng **chung** một bộ siêu tham số cho mỗi chế độ.

- **Lựa chọn epoch:** mô hình pretrained hội tụ nhanh nên 10 epoch là đủ (đường val gần như phẳng ở cuối, xem Hình 5). From scratch được 30 epoch, gấp ba. Với lịch cosine, val accuracy của from scratch cũng phẳng ở cuối, nhưng train accuracy đã gần 100%: mô hình bị giới hạn bởi overfitting nhiều hơn là bởi số epoch. Con số from scratch vì vậy là cận dưới so với một công thức huấn luyện dài và augmentation mạnh (Mục 8). Learning rate backbone khi fine-tune nhỏ hơn head 10 lần để không phá đặc trưng pretrained, trong khi head mới khởi tạo vẫn học nhanh.
- **Chọn mô hình:** lưu checkpoint có validation accuracy cao nhất, sau đó đánh giá **một lần** trên test.
- **Metric:** top-1 accuracy, top-5 accuracy, macro-F1, accuracy cấp superclass (dự đoán đúng superclass), confusion matrix 100×100.
- **Chi phí:** số tham số tổng/trainable, GMACs, thời gian train (tổng thời gian các epoch, không tính validation), thông lượng train/inference (ảnh/s), bộ nhớ GPU đỉnh khi train (`torch.cuda.max_memory_allocated`).
- **Phần cứng:** NVIDIA RTX 3050 Laptop 6 GB (65 W), Intel i7-13620H, 16 GB RAM, Windows 11; PyTorch 2.5.1 + CUDA 12.4, timm 1.0.30.

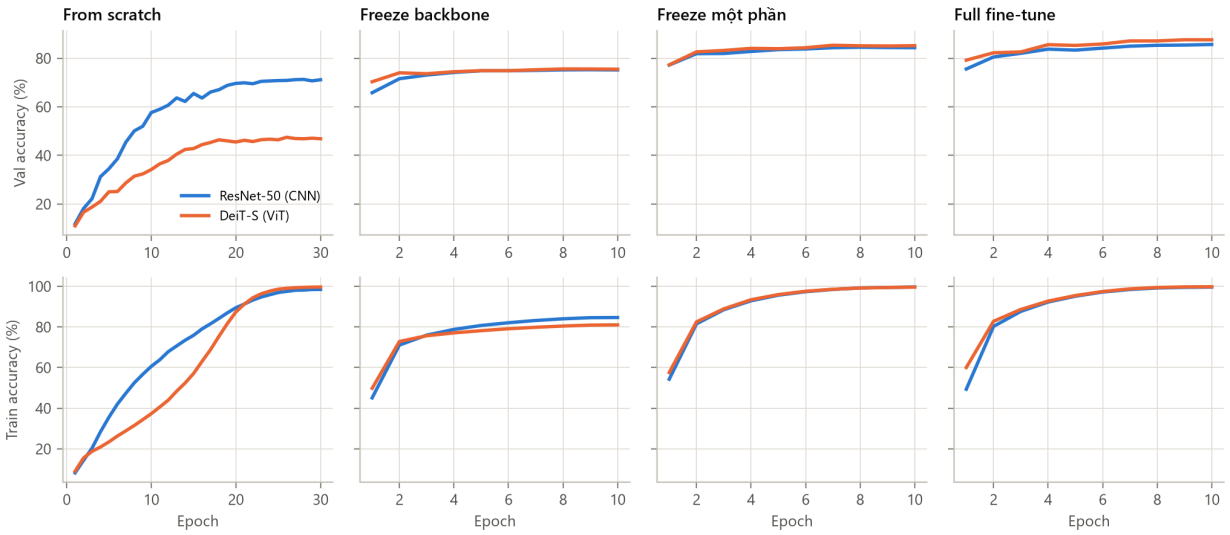
6 Kết quả

Model	Chế độ	Top-1	Top-5	Macro-F1	Super-class	Trainable (M)	Train (phút)	Train (ảnh/s)	Mem (GB)
ResNet-50	From scratch	71,37	90,48	71,27	80,80	23,71	100,4	224	3,26
DeiT-S	From scratch	48,26	71,55	47,76	59,96	21,70	85,5	263	2,62
ResNet-50	Freeze backbone	75,19	94,62	75,17	84,37	0,20	10,9	687	0,64
DeiT-S	Freeze backbone	74,52	94,28	74,51	84,53	0,04	8,6	870	0,45
ResNet-50	Freeze một phần	85,53	97,15	85,53	92,57	22,27	19,1	392	1,29
DeiT-S	Freeze một phần	85,26	96,92	85,22	92,12	10,69	18,6	404	1,48
ResNet-50	Full fine-tune	86,01	97,66	86,03	92,74	23,71	33,4	225	3,26
DeiT-S	Full fine-tune	88,57	97,94	88,60	94,46	21,70	28,8	261	2,62

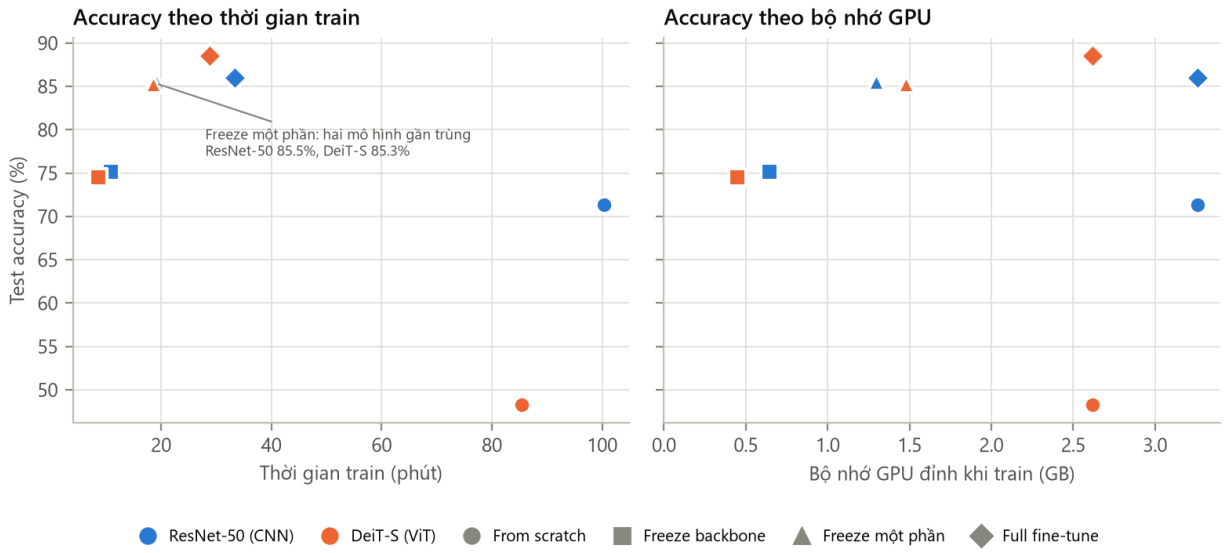
Bảng 6: Kết quả trên tập test CIFAR-100 (10.000 ảnh, %), checkpoint chọn theo validation. Chi phí đo trên RTX 3050 Laptop, batch 64, 224×224.



Hình 4: Top-1 accuracy trên test theo chế độ huấn luyện.



Hình 5: Đường học: validation accuracy (trên) và train accuracy trên batch đã augment (dưới). Trục x của from scratch dài gấp ba.



Hình 6: Accuracy theo thời gian huấn luyện và theo bộ nhớ GPU đỉnh. Màu là họ mô hình, hình dạng điểm là chế độ huấn luyện. Số tham số trainable có trong Bảng 6.

6.1 Nhận xét chính

- **Pretrained + full fine-tune tốt nhất ở cả hai họ:** DeiT-S đạt 88,57%, ResNet-50 đạt 86,01%. Đây cũng là chế độ có validation accuracy cao nhất của mỗi họ, nên chọn theo validation hay theo test đều cho cùng kết luận.
- **Thứ hạng hai họ đảo chiều.** From scratch, ResNet-50 hơn DeiT-S 23,11 điểm; khi full fine-tune, DeiT-S lại hơn ResNet-50 2,56 điểm.
- **Pretraining quan trọng với ViT hơn nhiều so với CNN.** So với from scratch, full fine-tune tăng 14,64 điểm cho ResNet-50 nhưng tăng tới 40,31 điểm cho DeiT-S, dù from scratch được gấp ba số epoch và thời gian train dài gấp ~3 lần.
- **Chỉ một lớp tuyến tính trên đặc trưng pretrained đã thắng from scratch:** freeze backbone của DeiT-S (0,04 M tham số trainable, 8,6 phút) đạt 74,52%, cao hơn ResNet-50 from scratch (71,37%, 100,4 phút).
- **Freeze backbone** rẻ nhất nhưng thấp hơn full fine-tune 10,82 (ResNet-50) và 14,05 điểm (DeiT-S). **Freeze một phần** gần như bằng full fine-tune với ResNet-50 (kém 0,48 điểm) nhưng kém rõ hơn với DeiT-S (3,31 điểm).
- Dữ liệu cân bằng nên macro-F1 bám sát top-1 (chênh không quá 0,5 điểm); top-5 và superclass accuracy giữ đúng thứ hạng của top-1.

6.2 Chênh lệch CNN–ViT có ý nghĩa thống kê không?

Mỗi cấu hình chỉ chạy một seed, nên cần kiểm tra chênh lệch giữa hai họ có vượt mức nhiễu hay không. Hai mô hình được đánh giá trên **cùng** 10.000 ảnh test, nên dùng kiểm định McNemar (bản chính xác, dạng nhị thức) trên các ảnh mà đúng một trong hai mô hình đoán đúng.

Chế độ	Chỉ ResNet-50 đúng	Chỉ DeiT-S đúng	DeiT-S − ResNet-50	p
From scratch	2845	534	−23,11	< 0,001
Freeze backbone	984	917	−0,67	0,13
Freeze một phần	643	616	−0,27	0,46
Full fine-tune	412	668	2,56	< 0,001

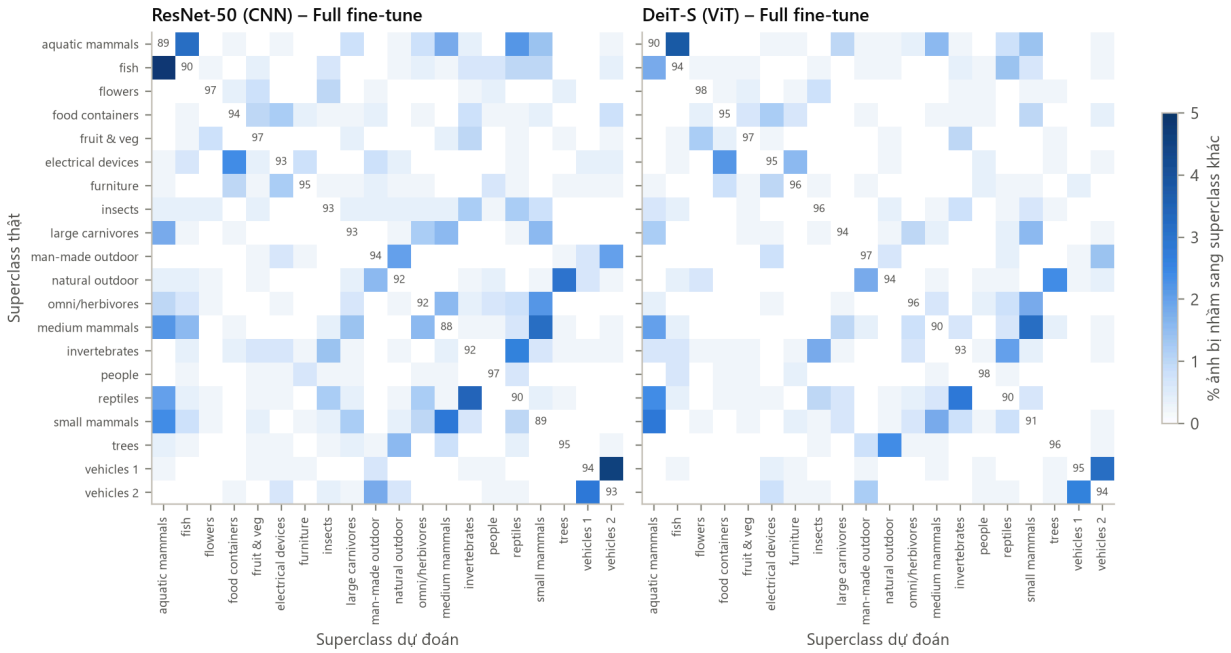
Bảng 7: Kiểm định McNemar giữa ResNet-50 và DeiT-S trong cùng chế độ (số ảnh test; chênh lệch tính bằng điểm %).

Chênh lệch ở **from scratch** và **full fine-tune** có ý nghĩa rất mạnh ($p < 0{,}001$). Ở **freeze backbone** ($p = 0{,}13$) và **freeze một phần** ($p = 0{,}46$), hai họ **không khác nhau có ý nghĩa**: với các chế độ đóng băng, đặc trưng ImageNet của hai backbone hữu ích gần như ngang nhau cho CIFAR-100.

6.3 Phân tích lỗi

Phần này phân tích chế độ tốt nhất (theo validation) của mỗi họ: full fine-tune.

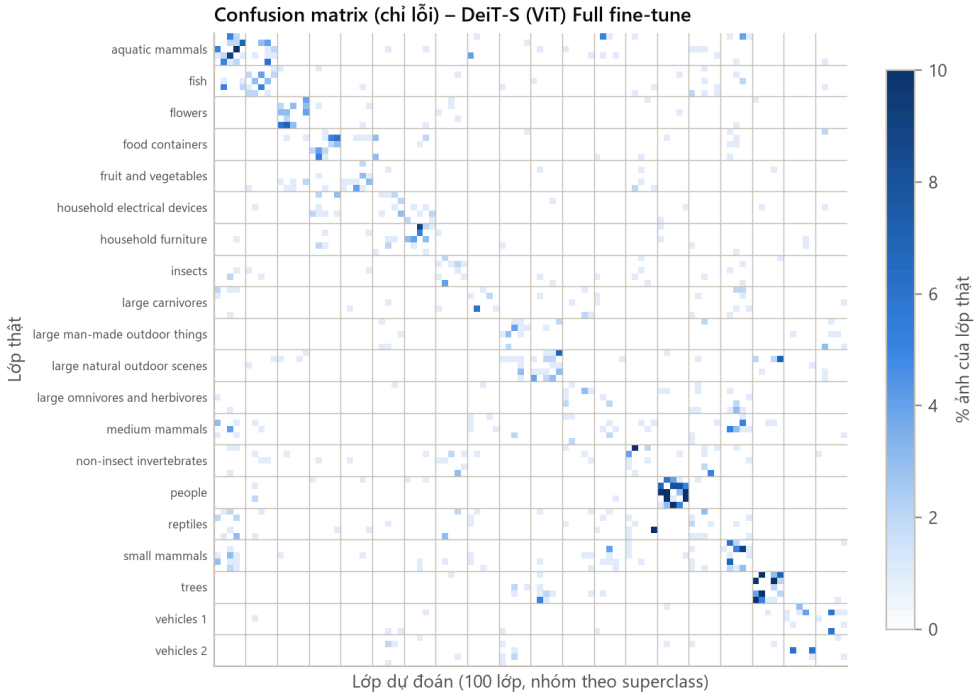
6.3.1 Lỗi theo superclass



Hình 7: Confusion matrix cấp superclass. Ô màu: % ảnh của superclass thật bị đoán sang superclass khác. Số trên đường chéo: accuracy của superclass đó.

- Accuracy cấp superclass đạt 92,74% (ResNet-50) và 94,46% (DeiT-S), cao hơn top-1 khoảng 6–7 điểm.
- **Hơn một nửa số lỗi là nhầm giữa các lớp "anh em" trong cùng superclass:** 51,5% lỗi của DeiT-S và 48,1% lỗi của ResNet-50. Nếu lỗi phân bố ngẫu nhiên thì tỉ lệ này chỉ là 4/99 ≈ 4%. Tỉ lệ này **tăng theo chất lượng mô hình**: 22,6% (DeiT-S scratch), 32,9% (ResNet-50 scratch), 37,0–39,3% (freeze backbone), 46,5–51,5% (freeze một phần, full fine-tune). Mô hình càng tốt thì lỗi còn lại càng "hợp lý" về ngữ nghĩa.
- Các cặp superclass hay nhầm nhất của hai mô hình gần như trùng nhau: **fish ↔ aquatic mammals** (cá mập/cá đuối với cá voi/cá heo), **vehicles 1 ↔ vehicles 2** (bus, streetcar, pickup truck, tractor… bị chia tùy ý vào hai nhóm), **medium → small mammals**, **reptiles → non-insect invertebrates** (snake → worm) và **large natural outdoor scenes → trees** (forest → cây). Đây là những cặp gần nhau về hình dạng hoặc do cách chia superclass của CIFAR-100 không hoàn toàn theo thị giác.
- DeiT-S from scratch có kiểu lỗi khác hẳn: nhầm vehicles 1 → vehicles 2 tới 14,2%, medium mammals → large carnivores 9,2%, food containers → household electrical devices 9,0%. Mô hình mới chỉ học được tín hiệu thô (màu, bố cục) chứ chưa học được cấu trúc ngữ nghĩa.

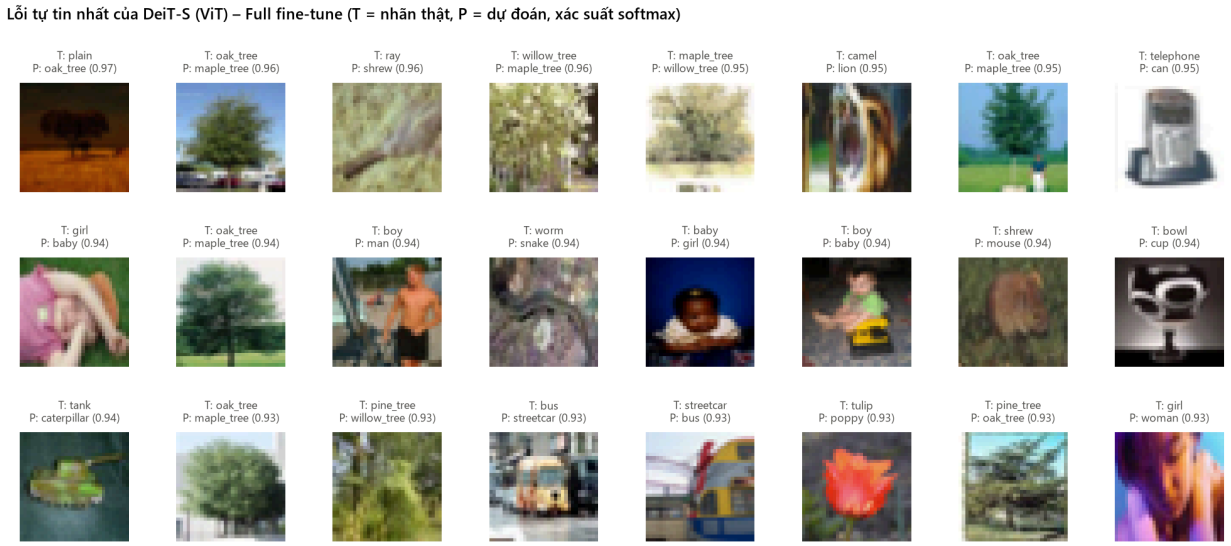
6.3.2 Lỗi theo lớp



Hình 8: Confusion matrix 100 lớp của DeiT-S full fine-tune, chỉ hiển thị phần lỗi (đường chéo để trống), lớp được nhóm theo superclass. Các khối đậm nằm trong các ô vuông trên đường chéo.

Lỗi tập trung thành vài khối rõ rệt trên đường chéo khối (Hình 8):

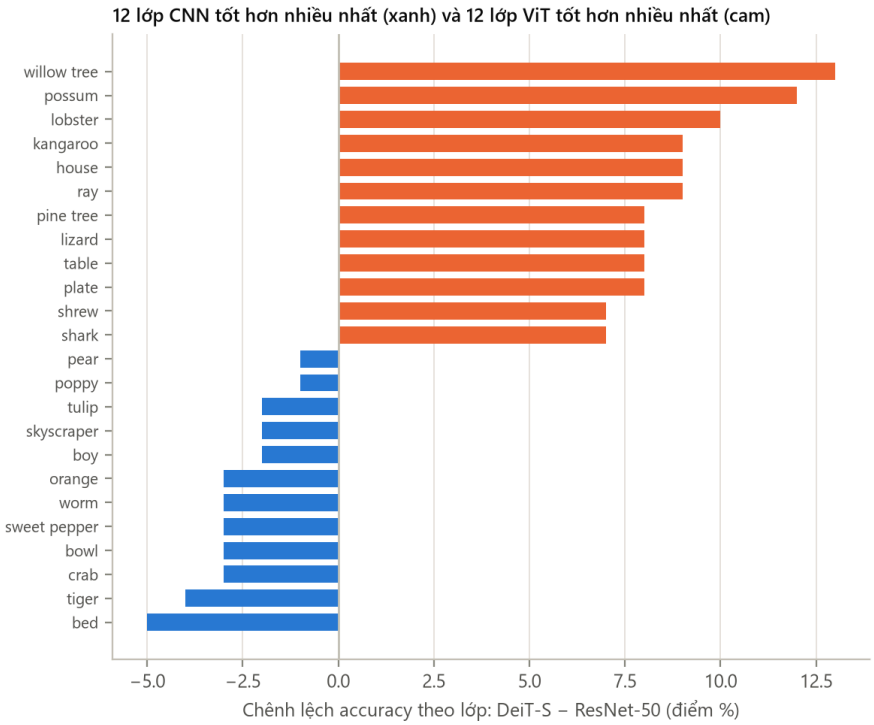
- **people** (baby, boy, girl, man, woman): lớp khó nhất của DeiT-S là girl (63%), tiếp theo là boy, man. Ở 32×32 khuôn mặt chỉ còn vài pixel, nên tuổi và giới tính phải đoán qua tóc, quần áo, bối cảnh.
- **trees** (maple, oak, palm, pine, willow): maple ↔ oak là cặp nhầm nhiều nhất của cả hai mô hình (DeiT-S: 17 + 15 ảnh; ResNet-50: 20 + 18 ảnh). Ở độ phân giải này hình dạng lá gần như mất hết, chỉ còn tán cây.
- **aquatic mammals** (otter ↔ seal) và **small/medium mammals** (shrew, possum, squirrel): động vật nhỏ màu nâu trên nền tự nhiên.
- Các lớp dễ nhất có hình dạng hoặc màu sắc đặc trưng: road (100%), tractor (98%), lawn mower (98%).



Hình 9: 24 lỗi có độ tự tin cao nhất của DeiT-S full fine-tune.

Xem các lỗi tự tin nhất (Hình 9), một phần đáng kể **không hẳn là lỗi của mô hình**: ảnh nhãn "plain" có một cái cây lớn ở giữa (đoán oak tree), ảnh nhãn "girl"/"boy" là trẻ nhỏ (đoán baby), ảnh "bowl" và "cup" rất giống nhau, nhiều ảnh cây chỉ là một tán lá mờ. Đây là giới hạn của nhãn đơn và độ phân giải 32×32, không cải thiện được chỉ bằng mô hình lớn hơn.

6.3.3 CNN và ViT khác nhau ở lớp nào?



Hình 10: Chênh lệch accuracy theo lớp giữa DeiT-S và ResNet-50 (full fine-tune): 12 lớp mỗi bên có chênh lệch lớn nhất.

DeiT-S thắng nhiều lớp với cách biệt lớn (willow tree +13, possum +12, lobster +10, kangaroo, house, ray +9 điểm), trong khi các lớp ResNet-50 thắng chỉ cách biệt tối đa 5 điểm (bed, tiger, crab, bowl, sweet pepper, orange). Các lớp DeiT-S thắng phần nhiều cần **hình dạng tổng thể** (dáng rủ của willow, kangaroo, ray, lobster); vài lớp ResNet-50 thắng có **kết cấu, màu sắc** đặc trưng (vằn tiger, orange,

sweet pepper). Điều này khớp với nhận định CNN thiên về texture và ViT thiên về shape (Geirhos và cộng sự, 2019; Naseer và cộng sự, 2021). Tuy vậy, mỗi lớp chỉ có 100 ảnh test, nên độ lệch chuẩn của hiệu accuracy một lớp vào khoảng 5 điểm. Chỉ các chênh lệch từ ~10 điểm trở lên mới đáng tin; phần còn lại nên xem là xu hướng.

6.3.4 Lỗi của hai họ bổ sung cho nhau

Trên tập test, hai mô hình full fine-tune cùng đúng 81,9%, cùng sai 7,3%, chỉ ResNet-50 đúng 4,1% và chỉ DeiT-S đúng 6,7%. Lấy trung bình xác suất softmax của hai mô hình (ensemble, không huấn luyện thêm) đạt **89,67%**, cao hơn mô hình tốt nhất 1,10 điểm. Cận trên nếu luôn chọn đúng mô hình (oracle) là 92,7%. Hai họ kiến trúc mắc lỗi đủ khác nhau để kết hợp có lợi.

7 Thảo luận

7.1 Pretrained so với from scratch

Với 45.000 ảnh train, cả hai mô hình from scratch đều overfit nặng (Hình 5): ResNet-50 có train accuracy 98,5% nhưng val chỉ ~71%; DeiT-S có train 99,7%, val ~47%. Ảnh 32×32 phóng lên 224×224 không thêm thông tin, nên mô hình ~22–24M tham số dễ dàng thuộc lòng tập train. Pretraining trên ImageNet-1k (1,28 triệu ảnh) cung cấp đặc trưng tổng quát mà 45.000 ảnh không đủ để học. Kết quả: full fine-tune hơn from scratch 14,64–40,31 điểm, với chỉ một phần ba số epoch, và val accuracy vượt 75% ngay sau epoch đầu tiên.

7.2 Khi nào ViT cần pretraining?

Trong thí nghiệm này: luôn cần. DeiT-S from scratch chỉ đạt 48,26%, thấp hơn ResNet-50 from scratch 23,11 điểm. Đường học cho thấy nguyên nhân không phải thiếu epoch: từ epoch 18, val accuracy của DeiT-S gần như đứng yên quanh 46–47%, trong khi train accuracy tăng từ 75% lên 99,7%. Mô hình đang học thuộc dữ liệu chứ không tổng quát hoá thêm.

Lý do nằm ở **inductive bias**. Convolution có sẵn tính cục bộ và tính bất biến tịnh tiến, nên ResNet-50 học được đặc trưng cục bộ hữu ích ngay cả với ít dữ liệu. Self-attention không có các giả định này: DeiT-S phải tự học từ dữ liệu rằng các pixel gần nhau thì liên quan, và với 45.000 ảnh nó học được rất ít. Điều này khớp với các nghiên cứu trước. ViT chỉ vượt ResNet khi được pretrain trên 14–300 triệu ảnh (Dosovitskiy và cộng sự, 2021). DeiT huấn luyện được ViT trên ImageNet-1k nhờ 300 epoch cùng augmentation và regularization rất mạnh (RandAugment, Mixup, CutMix, random erasing, stochastic depth) (Touvron và cộng sự, 2021). Protocol from scratch của bài này không có các yếu tố đó.

Khi đã được pretrain, ViT không còn bất lợi: cùng dữ liệu pretrain ImageNet-1k với ResNet-50, DeiT-S full fine-tune tốt hơn 2,56 điểm (có ý nghĩa thống kê, Bảng 7). Tóm lại, ViT cần pretraining khi dữ liệu nhỏ (cỡ vài chục nghìn ảnh) và công thức huấn luyện đơn giản; ngược lại, nếu có backbone pretrained thì ViT cạnh tranh tốt hoặc tốt hơn CNN cùng cỡ.

7.3 Khi nào đóng băng có lợi?

Freeze backbone (linear probe) nhanh hơn full fine-tune ~3,1–3,3 lần, dùng ít hơn 5 lần bộ nhớ, và chỉ phải lưu một head ~0,04–0,2 M tham số cho mỗi tác vụ. Đổi lại, accuracy thấp hơn 11–14 điểm. Ở đây mô hình **underfit chứ không overfit**: train accuracy trên ảnh đã augment chỉ 84,7% (ResNet-50) và 81,0% (DeiT-S), sát val. Đặc trưng lớp cuối của ImageNet không tách tuyến tính được hết 100 lớp CIFAR. Nguyên nhân là **domain gap**: ảnh CIFAR phóng to bị mờ, khác xa ảnh ImageNet sắc nét, và các đặc trưng sâu đã chuyên biệt cho 1000 lớp ImageNet. Freeze backbone có lợi khi tài nguyên rất

hạn chế, khi cần baseline nhanh, khi nhiều tác vụ dùng chung một backbone, hoặc khi dữ liệu quá ít để fine-tune mà không overfit.

Freeze một phần là điểm cân bằng tốt nhất cho ResNet-50: chỉ kém full fine-tune 0,48 điểm nhưng thời gian train giảm 43% và bộ nhớ giảm 60%. Hai stage đầu của CNN học các đặc trưng chung như cạnh, góc, kết cấu, vốn chuyển giao tốt giữa các tập dữ liệu (Yosinski và cộng sự, 2014). Các stage này lại chiếm phần lớn activation độ phân giải cao, nên bỏ qua backward ở đó tiết kiệm nhiều chi phí. Ngoài ra, layer3–layer4 vẫn được train và chứa 94% tham số.

Với DeiT-S, freeze một phần kém full fine-tune 3,31 điểm, mất nhiều hơn hẳn. Khi đóng băng, patch embedding và 6 block đầu (một nửa số tham số) giữ nguyên. Patch embedding là một phép chiếu tuyến tính học trên patch ImageNet sắc nét, và ở ViT mọi token đi qua cùng các block đó, nên phần đóng băng không thể thích nghi với patch mờ của ảnh CIFAR phóng to. Vì vậy, với ViT trên dữ liệu khác miền, nên fine-tune toàn bộ hoặc chỉ đóng băng rất ít block. Một lựa chọn trung gian là giảm learning rate dần theo độ sâu (layer-wise LR decay) thay vì đóng băng cứng.

Full fine-tune luôn cho accuracy cao nhất. Nó tốn nhiều chi phí nhất, nhưng trên GPU 6 GB vẫn vừa (3,26 GB với ResNet-50). Khi tài nguyên cho phép, đây là lựa chọn mặc định.

7.4 Chi phí

- **Thời gian và bộ nhớ:** trên GPU này, DeiT-S train nhanh hơn ResNet-50 (261 so với 225 ảnh/s khi full fine-tune), suy luận nhanh hơn (897 so với 705 ảnh/s) và tốn ít bộ nhớ hơn (2,62 so với 3,26 GB), dù GMACs cao hơn một chút (4,60 so với 4,09). Các phép nhân ma trận lớn của Transformer tận dụng Tensor Core với bfloat16 tốt hơn convolution và BatchNorm. Kết luận này phụ thuộc phần cứng và phiên bản thư viện.
- **Đường biên Pareto** (accuracy cao nhất ở mỗi mức thời gian, Hình 6) gồm freeze backbone, freeze một phần và DeiT-S full fine-tune. ResNet-50 full fine-tune bị DeiT-S full fine-tune vượt ở cả accuracy lẫn thời gian, còn hai cấu hình from scratch tốn nhiều thời gian nhất mà accuracy thấp nhất.

Tình huống	Lựa chọn đề xuất (theo kết quả trên CIFAR-100)
Cần accuracy cao nhất	DeiT-S pretrained, full fine-tune (88,57%); thêm ensemble với ResNet-50 nếu chấp nhận gấp đôi chi phí suy luận (89,67%)
Ngân sách train/bộ nhớ vừa phải	ResNet-50 hoặc DeiT-S freeze một phần (~85,4%, ~19 phút, ~1,3–1,5 GB)
Rất ít tài nguyên, nhiều tác vụ chung backbone	Freeze backbone (~75%, ~10 phút, < 0,7 GB)
Không có trọng số pretrained, dữ liệu nhỏ	CNN (ResNet-50 from scratch, 71,37%); tránh ViT from scratch nếu không có augmentation mạnh và lịch huấn luyện dài

Bảng 8: Tóm tắt khuyến nghị.

8 Hạn chế và hướng phát triển

- **Một seed duy nhất.** Ngân sách tính toán (~5,5 giờ cho 8 cấu hình trên GPU laptop) chỉ đủ cho một seed. Kiểm định McNemar (Bảng 7) giúp phân biệt chênh lệch thật với nhiễu trên tập test, nhưng không đo được phương sai do khởi tạo và thứ tự dữ liệu. Các chênh lệch dưới ~1 điểm nên được xác nhận bằng nhiều seed.
- **Công thức from scratch đơn giản.** Chỉ dùng 30 epoch, crop + flip và label smoothing; không có RandAugment, Mixup/CutMix, stochastic depth hay lịch dài hàng trăm epoch. Kết quả from scratch, nhất là của DeiT-S, là cận dưới. Hướng phát triển: công thức kiểu DeiT, hoặc distillation từ ResNet-50 sang DeiT-S qua distillation token để truyền inductive bias của CNN.

- **Độ phân giải 224 cho ảnh 32×32.** Phóng ảnh lên 224 giữ khớp với backbone pretrained nhưng tốn gấp ~49 lần tính toán so với 32×32, và mỗi patch 16×16 của DeiT-S chỉ ứng với ~2×2 pixel gốc. Với from scratch, ViT dùng patch nhỏ ở độ phân giải gốc (ví dụ patch 4 trên ảnh 32×32) là so sánh tự nhiên hơn.
- **Siêu tham số dùng chung, không tinh chỉnh riêng.** Không tìm learning rate riêng cho từng mô hình, không dùng layer-wise LR decay, vốn thường giúp fine-tune ViT tốt hơn.
- **"Một nửa" định nghĩa theo độ sâu**, nên số tham số trainable khác nhau (94% với ResNet-50, 49% với DeiT-S). Có thể bổ sung các mức đóng băng khác (theo tỉ lệ tham số hoặc FLOPs bằng nhau) để có đường cong accuracy theo mức đóng băng.
- **Phạm vi mô hình.** Chỉ một CNN và một ViT cùng pretrain trên ImageNet-1k có giám sát. Có thể mở rộng sang ConvNeXt, Swin, hoặc pretrain lớn hơn/tự giám sát (ImageNet-21k, DINO, MAE).
- **Đo thời gian trên laptop** có thể bị ảnh hưởng bởi nhiệt độ và tác vụ nền; con số thời gian chỉ nên dùng để so sánh tương đối giữa các cấu hình.

9 Tái lập kết quả

```
pip install -r requirements.txt --extra-index-url https://download.pytorch.org/whl/cu124
python -m src.eda                # thống kê + hình EDA
python -m scripts.run_all        # 8 cấu hình (bỏ qua run đã xong, resume run dở)
python -m src.analyze            # bảng tổng hợp, biểu đồ, phân tích lỗi
python -m scripts.build_report   # biên dịch báo cáo PDF này
```

Một cấu hình riêng: `python -m src.train --arch deit_s --mode full`. Mỗi run ghi `config.json`, `history.csv`, `metrics.json` và `preds.npz` (logits test) vào `results/<arch>_<mode>/`. Riêng số đo thời gian phụ thuộc phần cứng.

10 Tài liệu tham khảo

1. A. Krizhevsky. *Learning Multiple Layers of Features from Tiny Images.* Tech. report, University of Toronto, 2009.
2. K. He, X. Zhang, S. Ren, J. Sun. *Deep Residual Learning for Image Recognition.* CVPR 2016.
3. A. Dosovitskiy và cộng sự. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale.* ICLR 2021.
4. H. Touvron và cộng sự. *Training data-efficient image transformers & distillation through attention.* ICML 2021.
5. R. Wightman, H. Touvron, H. Jégou. *ResNet strikes back: An improved training procedure in timm.* 2021.
6. S. Kornblith, J. Shlens, Q. V. Le. *Do Better ImageNet Models Transfer Better?* CVPR 2019.
7. M. Raghu và cộng sự. *Do Vision Transformers See Like Convolutional Neural Networks?* NeurIPS 2021.
8. R. Wightman. *PyTorch Image Models (timm).* github.com/huggingface/pytorch-image-models.
9. R. Geirhos và cộng sự. *ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness.* ICLR 2019.
10. M. Naseer và cộng sự. *Intriguing Properties of Vision Transformers.* NeurIPS 2021.
11. J. Yosinski, J. Clune, Y. Bengio, H. Lipson. *How transferable are features in deep neural networks?* NeurIPS 2014.

12. Q. McNemar. *Note on the sampling error of the difference between correlated proportions or percentages.* Psychometrika, 1947.